

Model-Driven Safety Modeling and Analysis of Embedded Parallel Software

Yechun Jiang, Junda Ye, Huamin Qu
Software and Microelectronics Institute
Northwestern Polytechnical University
Xi'an, China
yechunjiang99@163.com

Xiaohan Wang
Troops 61336 of PLA
Beijing, China
Shengjun Ba
China Centre for Resources Satellite Data and
Application
Beijing, China

Abstract—this paper focuses on model-driven safety modeling and analysis of embedded parallel software. Firstly, analyze safety requirements of embedded parallel software and summarize potential safety risks because of software concurrency. Secondly, according to potential safety risks, extend AADL language which include increasing attribute of safety level, describing specific concurrency error type and error propagation to establish AADL safety model. Then use FMEA and Markov chain approaches to analyze and evaluate its safety. Finally, do an instance analysis with a flight control system software.

Keywords—embedded parallel software; AADL Model-driven; safety analysis; Markov chain

I. INTRODUCTION

On-chip multi-core processors (CMP) has become an effective solution to high performance embedded computing. It uses multiple processing core which has moderate performance to improve energy efficiency and using high task/thread level parallel to improve the whole performance of processor^[1]. However, when it bringing efficiency, many new safety risks are introduced due to synchronization, mutual exclusion and communication between the process/thread on multiprocessor^[2].

At present, the study of embedded parallel software mainly concentrated in parallel programming optimization^[3], formal modeling^[4], performance analysis^[5] and correctness validation^[6], etc. Safety analysis research for embedded parallel software is rarely. Model-driven software safety analysis has become a new trend. It mainly based on software architecture and components fault information, and can automatically construct fault tree and FMEA table, then analyze software safety^[7]. Such as FPTN^[8], CFTs^[9], SEFTs^[10], FPTC^[11] and Hip HOPS^[12]. Therefore, the research of embedded parallel software safety analysis is of great significance.

This paper focuses on model-driven safety modeling and analysis of embedded parallel software. Firstly, analyze safety requirements of embedded parallel software, and summarize potential safety risks because of software concurrency. Secondly, according to potential safety risks, extend AADL

language which include increasing attribute of safety level, describing specific concurrency error type and error propagation to establish AADL safety model. Then, use FMEA and Markov chain approaches to analyze and evaluate its safety. Finally, do an instance analysis with a flight control system software.

II. SAFETY RISKS OF EMBEDDED PARALLEL SOFTWARE

There are different potential safety risks under different embedded operating systems due to some reasons such as different scheduling mechanisms, resource allocation strategies. This part explores the potential safety risks due to the concurrency of embedded parallel software under VxWorks. We only consider the parallel between processes/thread of on-chip multi-core.

Under the multi-channel program environment, each process of system moves forward at the rate of unpredictable which results in non-reproducibility of results. To prevent this phenomenon, there are some restrictions among asynchronous processes, such as exclusive use and synchronization inter-process.

Embedded parallel software may introduce some new safety risks due to scheduling, synchronization, mutual exclusion, communications between process/thread, which are summarized as shown in Fig. 1, and it can be expanded according to the application domain. There are many studies about the scheduling, so it is not the focus of this paper.

III. SAFETY MODEL OF EMBEDDED PARALLEL SOFTWARE BASED ON AADL

AADL not only supports the modeling of software system's structure and behavior semantics, but also supports describing the system's non-functional properties and error models, so it has a good prospect in safety-critical embedded systems. After establishing safety model with AADL, we can do a safety analysis based on architecture level, where safety risks can be found in the early stages of software design, then give some suggestion to improve efficiency and ensure the safety of parallel software. Software safety model mainly contains the architecture model and error model.

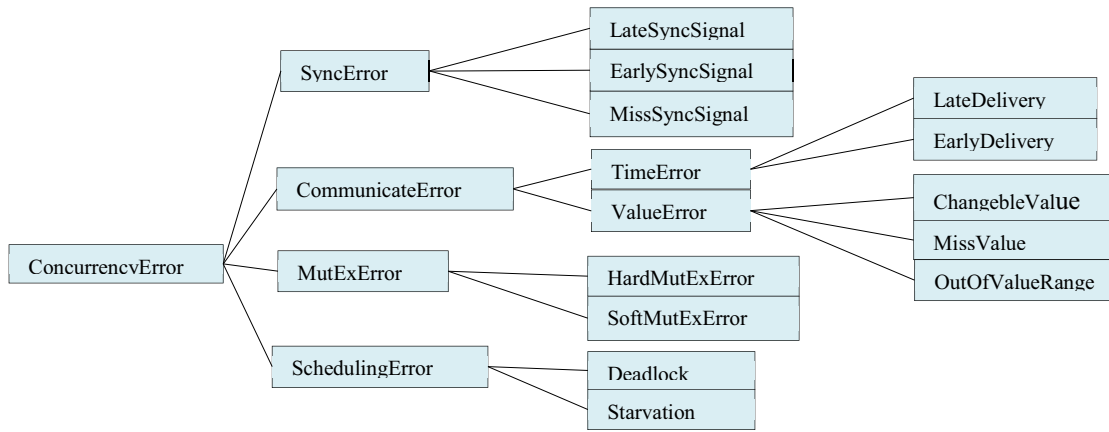


Figure 1. Safety risks of embedded parallel software

A. Architecture Model

AADL architecture model describes the hardware and software structure of the system. It describes the hardware and software architecture through components and connectors, depicts functions and non-functional properties through features and attributes, and illustrate the architecture evolution of runtime system through mode conversion, moreover, it use user-defined new attributes and annex to support scalability. In addition, we can use package to organize AADL files for complex systems modeling.

Fig. 2 and Fig. 3 are respectively AADL graphical and textual depiction of a simplified flight control system.

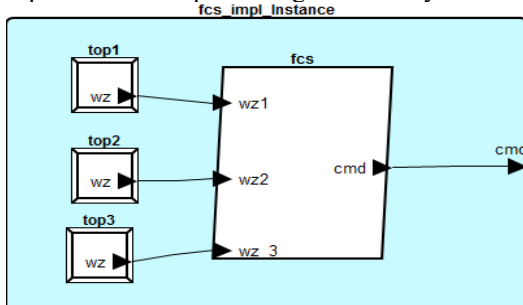


Figure 2. Graphical depiction of a simplified flight control system

```

system fcs
  features
    cmd : out data port;
    top1_failed:out event port;
    top2_failed:out event port;
    top3_failed:out event port;
  end fcs;

system implementation fcs.impl
  subcomponents
    fcs : process sw.impl;
    top1 : device top.impl;
    top2 : device top.impl;
    top3 : device top.impl;
  connections
    C1 : port top1.wz -> fcs.wz1;
    C2 : port top2.wz -> fcs.wz2;
    C3 : port top3.wz -> fcs.wz3;
    new_connection4 : port fcs.cmd -> cmd;
  end fcs.impl;

```

Figure 3. AADL textual description of a simplified flight control system

B. Extended Error Model

Error annex is a standard extension of AADL, mainly for modeling and describing safety of system and components. Error annex adopts behavioral state machine to describe system's random fault which may be caused by an internal error or external error propagation. State transitions in error model are triggered by error events and error propagation. Error events occurred in inside of components, and error propagation occurs between components.

1) Extended Definition of Error Type

AADL Error Annex organizes hierarchically and pre-declared the commonly used error types according to project experience. These pre-declared error types can be used to identify error events and error states. The common set of error types is provided as a standard error type library called ErrorLibrary.

Common set of error types called ErrorLibrary mainly state the following seven kind of error types. Namely ServiceError, TimingError, ValueError, RateError, SequenceError, ReplicationError and ConcurrencyError, as shown in Fig. 4.

```

fcs.aadl  *ErrorLibrary.aadl
Package ErrorLibrary
public
  annex EMV2{**
    error types
      ServiceError:type;
      TimingError:type;
      ValueError:type;
      RateError:type;
      SequenceError:type;
      ReplicationError:type;
      ConcurrencyError:type;
  **}
end ErrorLibrary;

```

Figure 4. Brief statements of common error types set ErrorLibrary

Now let's extend ConcurrencyError according to potential safety risks of embedded parallel software which summarized in previous part. Definition description as shown in Fig. 5.

After defined in package of MyErrorLibrary, these specific concurrency error types can be used directly in the files, as long as there is a statement "with MyErrorLibrary" at the beginning of the file.

```

package MyErrorLibrary
public
with ErrorLibrary;
annex EMV2{**
error types
ConcurrencyError: type; --并发错误
SyncError: type extends ConcurrencyError; --同步相关错误
MutExError: type extends ConcurrencyError; --互斥相关错误
CommunicateError: type extends ConcurrencyError; --通信相关错误
SchedulingError: type extends ConcurrencyError; --调度相关错误

EarlySyncSignal: type extends SyncError; --同步信号提前到达
LateSyncSignal: type extends SyncError; --同步信号延迟到达
MissSyncSignal: type extends SyncError; --同步信号丢失
SoftMutExError: type extends MutExError; --软件资源互斥
HardMutExError: type extends MutExError; --硬件设备互斥

ValueError: type extends CommunicateError; --值错误
OutOfValueRange: type extends ValueError; --值越界
MissValue: type extends ValueError; --值丢失
ChangebleValue: type extends ValueError; --值乱序

TimeError: type extends CommunicateError; --时间错误
EarlyDelivery: type extends TimeError; --提前到达
LateDelivery: type extends TimeError; --延迟到达

Deadlock: type extends SchedulingError; --死锁
Starvation: type extends SchedulingError; --饥饿

```

Figure 5. Specific type definition of ConcurrencyError

2) Description of Error Propagation

Sub-clauses error propagation used to define error propagation and error flows. It occurs when a component failure, and the failure can be observed by other components. Error propagation flows describe starting point, path and end point of error events by error source, error path and error sink. Error propagation of components as shown in Fig. 6.

```

annex EMV2
{**
use types MyErrorLibrary;
error propagations
ctl_navcalResult_in: in propagation {MissSyncSignal, ChangebleValue};
ctl_cticalResult_out: out propagation {ChangebleValue};
flows --define source of errors
f1: error path ctl_navcalResult_in {ChangebleValue} -> ctl_cticalResult_out;
f2: error sink ctl_navcalResult_in {MissSyncSignal};
end propagations;
**};

```

Figure 6. Error propagation description of components

3) Definition of Safety Level Attribute

ARP 4761 standard defines five safety levels according to the degree of harm caused by failure, namely Catastrophic, Hazardous, Major, Minor and NoEffect.

After defining safety levels and probability of occurrence as shown in Fig.7, we can get the probabilities of system in different safety levels by appropriate calculation and analysis.

```

properties
--MIL-STD 882D suggests descriptive labels
--(Catastrophic, Critical, Marginal, Negligible).
--ARP 4761 defines descriptive labels
--(Catastrophic, Hazardous, Major, Minor, NoEffect).
EMV2::Severity => 2 applies to s1_out.OutOfValueRange;

```

Figure 7. Definition of safety level attribute

IV. MODEL-DRIVEN SAFETY ANALYSIS OF EMBEDDED PARALLEL SOFTWARE

A. Improvement of Traditional Safety Analysis Methods

Traditional analysis methods like FTA and FMEA generate manually FTA table and FMEA table based on the experience of experts, which need higher requirements on

staff and ineffective. But now they can be generated automatically by OSATE with AADL safety model. It reduces a lot of tedious work and improve the work efficiency.

FEMA table generated automatically by OSATE, which recorded each error propagation path of each component. We can concluded the effects of component failure and the potential risks of system software by analyzing it. Then looking for the measures and improvement methods to improve the reliability and safety of software products.

B. Safety Analysis Based on Markov Chain

AADL model is a semi-formal model, in order to analyze accurately software safety, we need to convert AADL model into some formalized models for automatic calculation, such as Markov chain. Table I shows the elements mapping between AADL error annex and Markov chain model.

TABLE I. ELEMENTS MAPPING BETWEEN AADL ERROR ANNEX AND MARKOV CHAIN MODEL

Markov chain model	AADL error annex
S	Error state
Trigger Type	Error /Propagation
Probability	Occurrence
Q	Transitions

If a component has n error states, its model can be represented as a Markov chain transition matrix P which has n rows and n columns. P_{ij} represents the transition probability from state S_i to state S_j , which meets $0 \leq P_{ij} \leq 1$. $P_{ij} = 1$ means state S_i can only convert into state S_j , $P_{ij} = 0$ means state S_i can't convert into S_j . The sum of transition probabilities of the same initial state is 1, namely the sum of each row in P is 1.

Based on ergodicity of homogeneous Markov chain, its probability of each state will be stable after limited number of state transitions. So after a long time, we can calculate stable probability of component in each error state according to (1). π represents the probabilities of component in each state in S. P represents the transition matrix, π_i represents the probability that the component in state i.

$$\pi = \pi * P \quad (1)$$

$$\pi_i = \sum_{j=1}^{j=n} \pi_i P_{ji} \quad (2)$$

In order to analyze the safety of components, first we should calculate the stable probabilities of component in each error state according to (1) and (2), then we can get the probabilities of component in each safety level by calculate the sum of state probabilities of component in same safety level just as (3) presents. P_L represents the probability of component in state whose safety level is L.

$$P_L = \sum_{i, level=L} \pi_i \quad (3)$$

AADL error annex can describe composition of fault behavior and define the system's state with the composition of its internal components' state. There are four state combinational logic in AADL error annex, respectively are or, and, ormore, orless. Or means as long as either of conditions is satisfied, and means all of conditions must be satisfied, n ormore means n or more conditions are satisfied, n orless means n or less conditions are satisfied.

Thus, we can calculate the probabilities of system in each state according to state probabilities of components and

combinational logic. Then get the probabilities of system in each safety level by calculate the sum of state probabilities of system in same safety level. For example, assume the system state S1 is defined as S1 when 1ormore (sub1.f1, sub2.f2, sub3.f3), so the probability of system in state S1 can be calculated according to (4).

$$P_{S1} = 1 - \overline{P_{sub1.f1}} * \overline{P_{sub2.f2}} * \overline{P_{sub3.f3}} \quad (4)$$

Composition of states can avoid explosion of states which caused by mapping each state of each component to states of system directly, thus improving the efficiency of safety analysis.

V. VERIFICATION OF EXAMPLE

A. Introduction of Flight Control System

The flight control software is safety critical, whose safety level is the highest A according to the DO-178B/C, and its failure will directly affect flight safety. Therefore, we use the flight control system as an example to illustrate the process of model-driven safety modeling and analysis in this paper.

Flight control software is central pivot of the flight control system. It is responsible for real-time collection and appropriate processing of data for each sensor signal, and according to the designed navigation and guidance control algorithms to calculate real-time actuator output, then control the corresponding rudder deflection to achieve flight controls for different flight phases of aircraft.

B. Safety Modeling of Flight Control System

Flight control system often adopts triple redundancy management, here we only model for a set of components for simplification. Fig. 8 illustrate the AADL graphical description of the flight control system. Fig. 9 presents the inside structure of process component Nav_Ctl_Process.

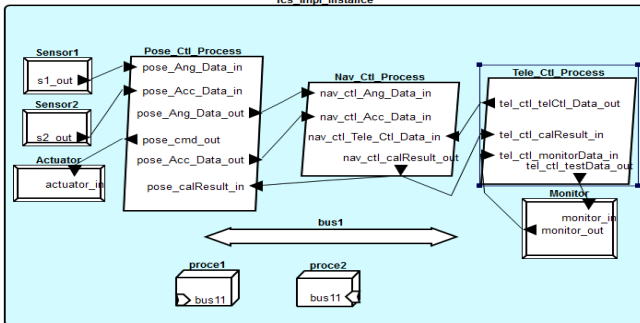


Figure 8. AADL graphical description of the flight control system

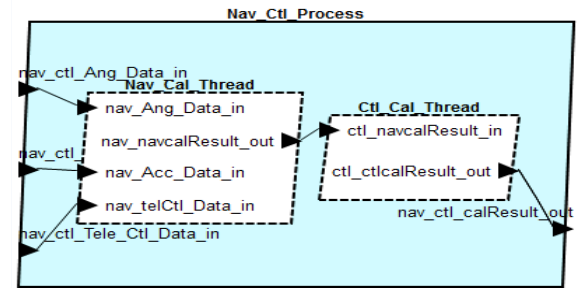


Figure 9. AADL graphical description of Nav_Ctl_Process

After establishing AADL architecture model, we also need to inject fault of components to get extended error model by defining error types and description of error propagation. Error model of Ctl_Cal_Thread as shown in Fig. 10. We only consider concurrent error types which has been defined in previous part, so we need not repeated definition but use statement "use types MyErrorLibrary".

```
thread Ctl_Cal_Thread
features
  ctl_navcalResult_in:in data port;
  ctl_cticalResult_out:out data port;
annex EMV2
{**
  use types MyErrorLibrary;
  error propagations
    ctl_navcalResult_in: in propagation {MissSyncSignal,ChangebleValue};
    ctl_cticalResult_out:out propagation {ChangebleValue};
  flows --define source of errors
    f1: error path ctl_navcalResult_in{ChangebleValue}->ctl_cticalResult_out;
    f2: error sink ctl_navcalResult_in{MissSyncSignal};
  end propagations;
**};
end Ctl_Cal_Thread;
```

Figure 10. Error propagation description of Ctl_Cal_Thread

C. Safety Analysis of Flight Control System

1) Safety Analysis Based on FMEA

The FMEA table generated automatically by OSATE tool based on safety model of instance as shown in Fig. 11. All failure modes of each component and the failure propagation path in the system can be obtained from the table, then to analyze the effects of each failure on the system.

2) Safety Analysis Based on Markov Chain

Suppose Fig. 12 illustrates the transitions between error states of Ctl_Cal_Thread. There are four states in the error model of Ctl_Cal_Thread, and here $\lambda_1 = 0.0000125$, $\lambda_2 = 0.25$, $\lambda_3 = 0.35$, $\lambda_4 = 0.23$.

Fault Impact of System Internal Error Sources										
Component	Initial	1st Leve	Failure	second I	Failure	third Le	Failure	4th Leve	Failure	5th Leve
Nav_Ctl_Process.N	{MissSync	{MissSync	Nav_Ctl_Process.Ctl_Cal_Thread	{MissSyncSignal}	[Masked]					
Nav_Ctl_Process.N	{Changebl	{Changebl	Nav_Ctl_	{Changebl	Pose_Ctl	{Changebl	Actuator	{ChangebleValue}	[Masked]	
Sensor1	{OutOfVal	{OutOfVal	Pose_Ctl	{OutOfVal	Nav_Ctl_Process.Nav_Cal_Thread	{OutOfValueRange}	[Masked]			

Figure 11. FEMA table generated automatically by OSATE

The first step is converting the AADL safety model into a Markov chain model. Map component Ctl_Cal_Thread's states namely Error_free, Erroneous, Fault, and Failed to states 1 to 4 respectively. Then obtain the initial state probabilities of Ctl_Cal_Thread $\pi = \{1, 0, 0, 0\}$, and according

to the error model description to get the transition matrix P as show in (5). State probabilities of component will reach a stable state after some time. Write a program according to (1) and (5) to obtain stable probabilities π of Ctl_Cal_Thread in each state After running, get $\pi = \{0.999938, 1.24992e-005,$

8.92802e-006, 4.07583e-005}. If only state Failed is catastrophic, so the probability of component Ctl_Cal_Thread in catastrophic level is 4.07583e-005.

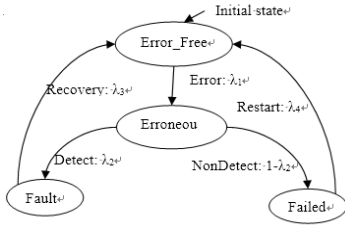


Figure 12. States transition of Ctl_Cal_Thread

Calculate state probabilities of each component, then calculate the state probabilities of flight control software in NoEffect, Minor, Major, Hazardous and Catastrophic respectively.

$$P = \begin{bmatrix} 0.9999875 & 0.0000125 & 0 & 0 \\ 0 & 0 & 0.25 & 0.75 \\ 0.35 & 0 & 0.65 & 0 \\ 0.23 & 0 & 0 & 0.77 \end{bmatrix} \quad (5)$$

For example, flight control software has three states S1, S2, S3. And through composition of states S1 is defined as S1 when $\text{Iormore}(\text{Pos_Ctl_Process.failed}, \text{Nav_Ctl_Process.failed}, \text{Tele_Ctl_Process.failed})$, so the probability of system in state S1 can be calculated according to (6).

$$P_{S1} = 1 - \overline{P_{PCP.failed}} * \overline{P_{NCP.failed}} * \overline{P_{TCP.failed}} \quad (6)$$

We can calculate the probabilities of system each safety level according to (3), then do the safety analysis.

VI. CONCLUSION

This paper focuses on model-driven safety modeling and analysis of embedded parallel software. Firstly, analyze safety requirements of embedded parallel software, and summarize potential safety risks because of software concurrency. Secondly, according to potential safety risks, extend AADL language which include increasing attribute of safety level, describing specific concurrency error type and error propagation to establish AADL safety model. Then, use FMEA and Markov chain approaches to analyze and evaluate its safety. Finally, do an instance analysis with a flight control system software.

ACKNOWLEDGMENT

This work is supported by the Aeronautical Science Foundation of China (20120718005), Astronautical Supporting Technology Foundation of China (2014HTXGD) and the Basic Research Foundation of Northwestern Polytechnical University (in China).

REFERENCES

[1] Qian Yu, Tong Li, Zhong Wen Xie, Na Zhao, Ying Lin. Distributed Computing Design Methods for Multicore Application Programming. In proceedings of 2012 International Conference on Computer Science and Communication Technology (CSCT 2012), IEEE Computer Society.

[2] Jun-hua H, Wei Z. Method for modeling and analysis real-time system dependability using AADL[C]//Electric Information and Control Engineering (ICEICE), 2011 International Conference on. IEEE, 2011: 3356-3359.

[3] YUAN ChengZhi, WANG Cong. Design and performance analysis of deterministic learning of sampled-data nonlinear systems[J]. Science China(Information Sciences), 2014, 03: 204-221.

[4] Hongping Zhang, Yongning Wen, Jie Jiang, Wei Huang. INDOOR 3D ROUTE MODELING BASED ON ESTATE SPATIAL DATA[A]. ISPRS Technical Commission IV Symposium, 2014: 6.

[5] Complexity of synthesis of composite service with correctness guarantee[J]. Science China(Information Sciences), 2012, 03: 638-649.

[6] Rugina A E, Kanoun K, Kaâniche M. The ADAPT tool: From AADL architectural models to stochastic petri nets through model transformation[C]//Dependable Computing Conference, 2008. EDCC 2008. Seventh European. IEEE, 2008: 85-90.

[7] Boudali H, Crouzen P, Stoelinga M. A rigorous, compositional, and extensible framework for dynamic fault tree analysis[J]. Dependable and Secure Computing, IEEE Transactions on, 2010, 7(2): 128-143.

[8] Rugina A E, Kanoun K, Kaâniche M. A system dependability modeling framework using AADL and GSPNs[M]//Architecting Dependable Systems IV. Springer Berlin Heidelberg, 2007: 14-38.

[9] Ramaiah B S M P S, Gokhale A A. FMEA and Fault Tree based Software Safety Analysis of a Railroad Crossing Critical System[J]. Global Journal of Computer Science and Technology, 2011, 11(8).

[10] Yang ZB, Pi L, Hu K, Gu ZH, Ma DF. AADL: An architecture design and analysis language for complex embedded real-time systems[J]. Journal of Software, 2010, 21(5): 899-915.

[11] Bomel P, Blouin D, Lanoe M, et al. Functional validation of AADL models via model transformation to SystemC with ATL[C]//Proceedings of the 5th International Workshop on Model Based Architecting and Construction of Embedded Systems. ACM, 2012: 13-18.

[12] Boudali H, Crouzen P, Stoelinga M. A rigorous, compositional, and extensible framework for dynamic fault tree analysis[J]. Dependable and Secure Computing, IEEE Transactions on, 2010, 7(2): 128-143.

[13] Bomel P, Blouin D, Lanoe M, et al. Functional validation of AADL models via model transformation to SystemC with ATL[C]//Proceedings of the 5th International Workshop on Model Based Architecting and Construction of Embedded Systems. ACM, 2012: 13-18.

[14] Munoz M. Space systems modeling using the Architecture Analysis & Design Language (AADL)[C]//Software Reliability Engineering Workshops (ISSREW), 2013 IEEE International Symposium on. IEEE, 2013: 97-98.

[15] Mishra A, Rao M, CU C, et al. An auto-review tool for model-based testing of safety-critical systems[C]//Proceedings of the 2013 International Workshop on Joining AcadeMiA and Industry Contributions to testing Automation. ACM, 2013: 47-52.

[16] Bin Gu, Yunwei Dong, Xiaomin Wei. A Qualitative Safety Analysis Method for AADL Model. Software Security and Reliability-Companion (SERE-C), 2014 IEEE Eighth International Conference on IEEE, 2014: 213-217.

[17] Xiaomin Wei, Yunwei Dong, Mengmeng Yang, Ning Hu, Hong Ye. Hazard analysis for AADL model. Embedded and Real-Time Computing Systems and Applications (RTCSA), 2014 IEEE 20th International Conference on, 20-22 Aug. 2014: 1-10

[18] Delange, J, Feiler, P. Architecture Fault Modeling with the AADL Error-Model Annex. Software Engineering and Advanced Applications (SEAA), 2014 40th EUROMICRO Conference on IEEE, 2014: 361-368.

[19] Quan Zhang, Shihai Wang, Bin Liu. Some Improvements on the Rules for Exchanging Between Error Model Annex and AADL to Fault Tree. Information Technology and Applications (ITA), 2013 International Conference on IEEE 2013: 338-342.